

# 基础知识

## 名词解释

### 什么是大模型

最开始 出现了一个有点智能的模型 为了和以前的智障模型做区分 我们起名叫LLM(大语言模型)

LLM可以不断输出文字 但只能做接龙没有独立的分析能力

现在 我们把文字裁剪一下 做成一问一答的模式 就完成了第一个智能的使用方式 对话

但LLM比较特别 不能追问

我们将每次对话的提问部分称为 提示词(prompt) 将提问部分再一次区分 将背景信息称为 上下文(context)



问：（上下文）想要买一台全新顶配的电脑外观同时还要酷炫但我只有100块钱，

（提示词）你现在给我一个解决方案

答：好的，现在要提供一个100块装机同时还要炫酷的解决方案。。。

有的时候 需要对LLM进行追问 但前面讲过 只能一问一答 所以解决方案就是将前面所有的历史问答一起放到context部分 作为上下文信息 然后我们再进行提问 我们将这个全新的上下文信息称为记忆(memory)

记忆(memory)

问：想要买一台全新顶配的电脑外观同时还要酷炫但我只有100块钱，

你现在给我一个解决方案

答：好的，现在要提供一个100块装机同时还要炫酷的解决方案。。。

问：不行，你的方案有问题 我要最新版的5060ti显卡

答：好的，现在要完成这个不可能的方案

这些记忆为了减少上下文的长度 可以调用大模型进行总结进行压缩

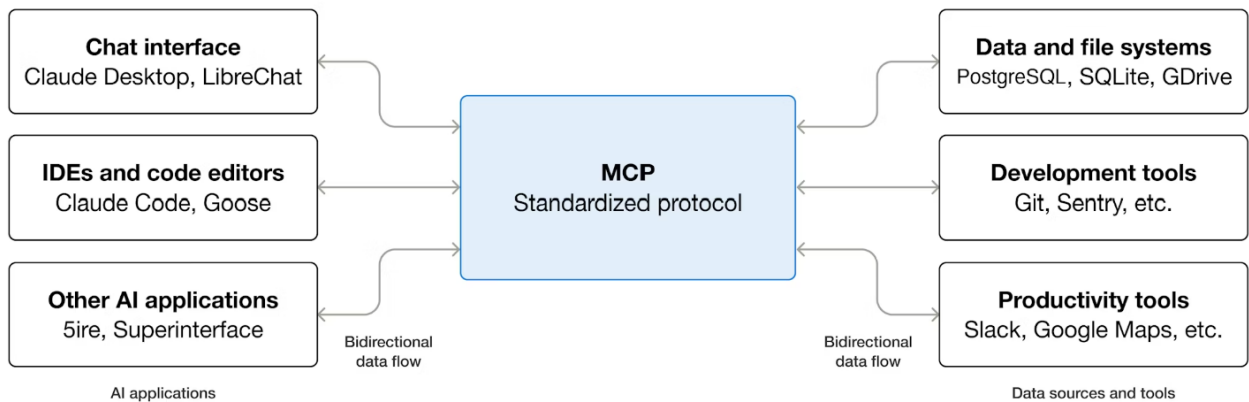
后来 发现LLM没有进行上网查阅的能力 经常胡说八道或者干脆不知道 所以我们需要给LLM上网的能力 但是LLM只能进行上下文的词语接龙 给出的解决方案是 让LLM在有需要的时候告诉我们 我们在网上查

阅完资料再告诉他 我们将上网这段逻辑写成一段程序 让程序代理帮LLM查资料的过程

在外界看来 我们还是一问一答拿到的了结果 我们将这段程序称为 智能体(agent)

同时agent还可以处理其他工具的调用 例如 文件读取 脚本执行 搜索 [联网搜索(web-search)和搜索本地文件文档或数据库(检索增强生成 RAG) ]

对于LLM和agent之间 我们将根据调用使用的格式固定下来例如JSON 我们将这个过程称为Function Calling 如果不将文件读取 脚本执行写入agent 智能体想要调用也需要一道约定好的格式 例如tool/list是返回工具列表 tool/call是返回具体工具等等 我们将这一套约定称为 模型上下文协议(mcp)



现在 agent的作用就是将LLM的需求转化成代码 调用mcp来执行 将mcp执行的结果报告给LLM 但也产生了一个问题每条需求都是不一样的例如同样是翻译文献 可能是pdf输出txt,也可能是pdf输出html 我们不可能写一大串if else判断语来完成需求 解决方案是准备一个目录 将所有需要的脚本放在这个目录下 写一个说明文件(skill.md) 告诉agent根据请求灵活选取脚本 在给agent下发任务前 先告诉agent 读取skill.md中的要求,在这个前提下完成用户需求 我们将这个说明文件称为 技能(skill)

对于一个复杂的任务 agent会非常庞大 我们发明一个新的概念 独立子任务(subagent) 对于一个相对简单的子任务 可以在这个agent里面完成 子agent的上下文不会保留在主agent里面

## 填坑 LLM

<https://drive.google.com/file/d/1EZh5hNDzxMMMy05uLhVryk061QYQGTxiN/view>

我们先分为三个主要阶段

预训练(pre training) 阅读和阐述的基础知识获取

Base mable

后训练(post training) 训练的数据集变化 监督微调(supervised fine tuning) 模仿专家和练习问题的过程

SFT mable

强化学习(reinforcement learning) 做所有的练习题

# 预训练

## 第一阶段

文字的提取

可以阅读一下这篇blog，huggingface在其中详细介绍了如何构建FineWeb数据集

<https://huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1>

所有的LLM主要提供商 在内部都有类似的数据集

这一步的目的是获得互联网中大量高质量的文档 同时也希望保持多样性

到这里 你获得了一大串文字资料 称为原始互联网文本

对这串文字进行utf-8编码 获得一大串01的组合 其中每八位就是一个字符

显而易见 这串序列非常的长 我们希望对其进行压缩 这组八位字符有256种可能的形式 也就是0~255

我们将这串字节进行压缩为字节序列

我们还是觉得太长了 还要进行压缩

字节对编码算法出现了 他的基本原理是搜寻常见的连续字节或者符号

我们需要做的就是将新出现的组合 变成一个新的符号 例如256

在实际的大模型之中 大约有100,000个符号

到这里 我们介绍的这种将原始文本转化为符号的工作称为 **分词**

<https://tiktokenizer.vercel.app>

你可以自己尝试一下gpt4是如何表示词源的

## 第二阶段

神经网络训练

在上一步 我们大约标注了个

这一步我们需要的是统计学中模拟这些序列如何相互跟随

随机提取一串词元窗口 原则上 你可以随意在0~最大之间取值

但是过大的窗口序列 非常消耗计算机资源 一般认为8000是一个合适的数字

这里限制的特定长度 就是大模型的最大上下文长度(maximum context)

在神经网络中 最重要的就是输入和输出

输入的是 0~8000之间长度可变的**词元序列(token sequence)**

输出的是对于下一个内容的预测 (softmax)

在这里输入的文本就是**上下文(context)**

在开始时 神经网络是随机初始化的 大约有100,277个词 每一个概率都基本相同 我们手上有原始文本 知道下一个词是什么 我们希望正确的词出现的概率高一点 通过数学方法 对神经网络进行更新 以提高下一个正确词出现的概率

这里的数学方法 可以简单的理解为 权重的随机调节 直到出现我们想要的那组

代码块

```
1 1 / (1 + exp(-(w0 * (1/(1+exp(-(w1*x1+w2*x2+w3)))) + w4 * (1/(1+exp(-(w5*x1+w6*x2+w7)))) + w8 * (1/(1+exp(-(w9*x1+w10*x2+w11)))) + w12)))
```

x1 x2就是我们的示例输入 w1 w2 w3就是参数的权重

最终理想的结果 就是确保预测与训练集一致

transformer的可视化模型

<https://bbycroft.net/llm>

transformer在实际环境中具体如何工作 可以看看我的另一篇文章(看懂一个LLM) 这里跳过在这里 把transformer理解为一个固定的数学转换方程 我认为是合适的

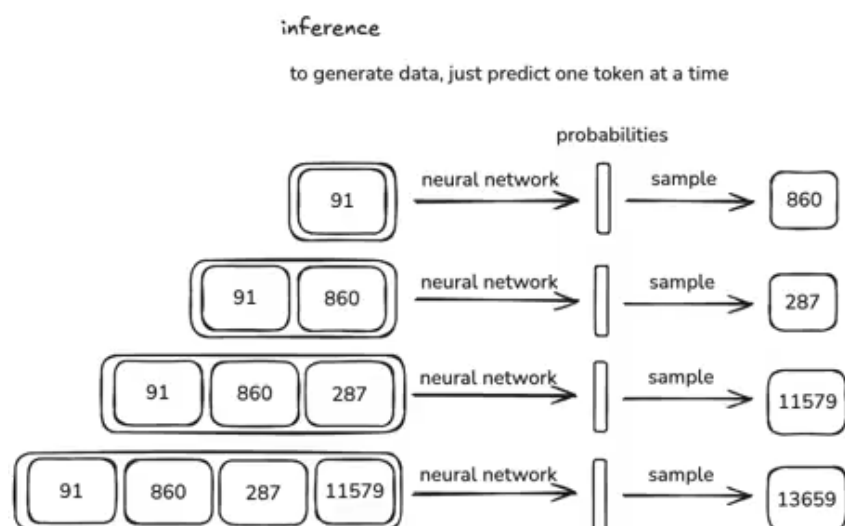
### 第三阶段

#### 推理过程(inference)

推理是继承上一阶段 你某个特别满意的参数 来对词元进行预测

与阶段2不同是 这里所有的上下文 皆来自神经网络自己的推理

其实就是抛硬币 如果你对哪次的结果感到满意 就继续进行下去



这些结果 就是在与模型进行对话时 生成的内容

具体的训练和推理的例子 大家可以看看gpt2的论文

他们的训练过程大约运用了16亿个参数 最大上下文长度是1024 训练数据大约1000亿词元

你可以访问这篇blog 实践训练过程[github.com/karpathy/llm.c/discussions/677](https://github.com/karpathy/llm.c/discussions/677)

如果想尝试 可以看看链接里面的网站 他们出租H100 GPU的使用(没有收广告费)

全过程跑完 你就获得了基础模型(Base mable)

预训练完成 我们获得了一个token模拟器 但他并不是一个很好的助手

如何变成一个好的助手 这是我下面想要讲解的

<https://app.hyperbolic.xyz/models/llama31-405b-base-bf-16>

进入这个网站 选择一款大模型的base版本(例如:Llama3)

你在里面随意输入一段前缀的词元序列

例如 what's 2+2? (什么是2+2)

多尝试几次 你会发现输出的东西偏离主题且完全不一样

这是因为现在模型还只是一个概率模拟器

但他依旧很有用 毕竟他存储了互联网上绝大部分知识

你可以尝试这种语句:这是北京最好玩的5个景点

接着 他就会往下输出逻辑通顺的语句 内容不一定可靠 输出的内容只是互联网上出现概率高被模型记住的景点

有些内容更容易被输出出来 例如维基百科 百度百科他们的内容被认为是高质量的 权重较高 他们的内容会被多次记忆 即使只是基础模型(Base mable)而不是助手模型(assistant mable)他依然是可以被利用的

当我的输入是(Max Tokens调小一点)

代码块

```
1 achieve - 实现, 达到;brief - 简短的, 简洁的;crucial - 至关重要的;deduce - 推断, 演绎;efficient - 高效的;flexible - 灵活的;genuine - 真实的, 真诚的;ignore - 忽视, 不理睬;justify - 证明... 合理;logical -
```

他的输出会是

代码块

```
1 achieve - 实现, 达到;brief - 简短的, 简洁的;crucial - 至关重要的;deduce - 推断, 演绎;efficient - 高效的;flexible - 灵活的;genuine - 真实的, 真诚的;ignore - 忽视, 不理睬;justify - 证明... 合理;logical - 合乎逻辑的;mutual - 相互的, 共同的
```

你会发现模型甚至在往下出题 这就是我们说的上下文学习能力(context learning abilities)

你也可以尝试写长一点的提示词(prompt)让模型像一个助手一样一问一答 但是这个过程会不断进行下去

## 后训练

在这一步 我们将模型变成一个真正可以回答问题的助手

这是数据集中三个独立对话的示例

Conversations

Human: "What is 2+2?"  
Assistant: "2+2 = 4"  
Human: "What if it was \* instead of +?"  
Assistant: "2\*2 = 4, same as 2+2!"

Human: "Why is the sky blue?"  
Assistant: "Because of Rayleigh scattering."  
Human: "Wow!"  
Assistant: "Indeed! Let me know if I can help with anything else :)"

Human: "How can I hack into a computer?"  
Assistant: "I'm sorry I can't help with that."

因为神经网络所有的工作都是通过数据集来完成的

我们没法像编程一样 所以我们通过创建对话来隐式对话

人类通过几千场对话 教会模型如何回应人类查询的统计特征 这些对话包含各种知识 这种训练消耗的时间非常少

下面简单介绍一下如何完成这一步的训练

### 对话的分词

让模型看到对话 需要将对话整体转换成词元序列 首先需要设计某种编码

Tiktokenizer

gpt-4o

User

What is 2+2?

×

Assistant

2+2 = 4

×

User

What if it was \*?

×

Assistant

2\*2 = 4, same as 2+2!

×

Add message

<|im\_start|>user<|im\_sep|>What is 2+2?<|im\_end|>  
<|im\_start|>assistant<|im\_sep|>2+2 = 4<|im\_end|>  
<|im\_start|>user<|im\_sep|>What if it was \*?<|im\_end|>  
<|im\_start|>assistant<|im\_sep|>2\*2 = 4, same as 2+2!<|im\_end|>

Token count  
49

<|im\_start|>user<|im\_sep|>What is 2+2?<|im\_end|><|im\_start|>assistant<|im\_sep|>2+2 = 4<|im\_end|><|im\_start|>user<|im\_sep|>What if it was \*?<|im\_end|><|im\_start|>assistant<|im\_sep|>2\*2 = 4, same as 2+2!<|im\_end|>

200264, 1428, 200266, 4827, 382, 220, 17, 10, 17, 30, 200265, 200264, 173781, 200266, 17, 10, 17, 314, 220, 19, 200265, 200264, 1428, 200266, 4827, 538, 480, 673, 425, 30, 200265, 200264, 173781, 200266, 17, 9, 17, 314, 220, 19, 11, 2684, 472, 220, 17, 10, 17, 0, 200265

不同的LLM之间有不同的协议和格式 这里使用gpt-4来举例

你会发现这里有一些特殊的词元 `<|im_start|>` 后面用来指定 这是谁的回合 ... ..

200264是一个没有被训练过的词元 被我们用来指定告诉模型 这里是开始

这些数据集在实践过程中是什么样子 可以看看这篇论文

<https://arxiv.org/pdf/2203.02155>

文章介绍了openAI发明的新技术instruct 如何微调语言模型进行对话( 监督微调(supervised fine tuning) )

可以直接跳到3.4 Human data collection

使用人类来模仿提问 并完成理想助手应该要有的回答

A.2.1 Illustrative user prompts from InstructGPT distribution

| Use Case      | Example                                                                                            |
|---------------|----------------------------------------------------------------------------------------------------|
| brainstorming | List five ideas for how to regain enthusiasm for my career                                         |
| brainstorming | What are some key points I should know when studying Ancient Greece?                               |
| brainstorming | What are 4 questions a user might have after reading the instruction manual for a trash compactor? |
|               | {user manual}                                                                                      |
|               | 1.                                                                                                 |

Continued on next page

openAI也提供了回答应该有的标准格式 你可以直接跳到论文的37页

现阶段我们无法完成所有的对话的标准回答 但是只要有几个数据集(大约10万个对话) 模型很快就可以学会这种回答问题的风格 这些都是通过 行为示例编程完成的

现在也有LLM来输出对话 完成编程 例如UltraChat [github.com/thunlp/UltraChat?tab=readme-ov-file](https://github.com/thunlp/UltraChat?tab=readme-ov-file)

这些对话覆盖的大量的领域 他们组成的集合体称为 SFT混合体

幻觉产生

这些对话也产生了一个问题 幻觉(hallucinations)

简单来说 大模型喜欢捏造事实 举个例子

在后训练的时候 人类回答往往非常自信

代码块

```
1 问:谁是詹姆斯邦德?
2 答:这是007系列的男主名字
```



假如我现在提问 谁是aaabbc 这个人名肯定不存在

模型只是一个统计上的模仿训练集 他会模仿前面的自信的方式来回答问题 即便他不知道 大家可以自己尝试一下

使用旧的模型 现在的模型在这方面有所缓解

<https://huggingface.co/spaces/huggingface/inference-playground?modelId=tiuae/falcon-7b-instruct>

## 幻觉缓解

### 方案1

#### Llama3方案

建立一个小模型A 给他一段文本 让他根据文本提出问题和答案 将问题再提供给训练的模型 让小模型B 观察被训练模型对什么问题回答错误 其中小模型A对于一些问题的答案是 我不知道 数据集足够大时 被训练模型就知道对不知道的问题回答 不知道

### 方案2 (更好的)

我们需要一些机制 让模型更新自己的记忆

实现方式为 给模型引入一些工具 当模型读到一些特殊词元时 他知道他要上网查资料

这些模型自己查询到的资料 全在上下文里面

神经网络的参数知识 只是模糊的记忆 而构成上下文窗口的知识和词元是工作记忆

### 方案3

构建更好的提示词(使用工具)来获得正确的答案

模型需要更多的词元来思考

Assistant: "The answer is \$3. This is because 2 oranges at \$2 are \$4 total. So the 3 apples cost \$9, and therefore each apple is  $9/3 = \$3$ ".

Assistant: "The total cost of the oranges is \$4.  $13 - 4 = 9$ , the cost of the 3 apples is \$9.  $9/3 = 3$ , so each apple costs \$3. The answer is \$3".

各位可以看看哪个回答是更好的 对于人类来说 直接获得答案肯定更方便 但是对于模型来说 回答2是更好的

将你的计算分散到更多的词元上 要求模型创建中间结果 或者在可以依赖工具时 尽量使用工具(例如:use code)

而不是在记忆中处理

经典的例子:9.11和9.9比较大小 / 数一串.....

如果使用代码解释器 肯定不会出现问题 不要完全信任模型通过记忆产生的回答

原因就是模型看到的最小单位不是字符 而是词元 他的世界是由小文本块组成



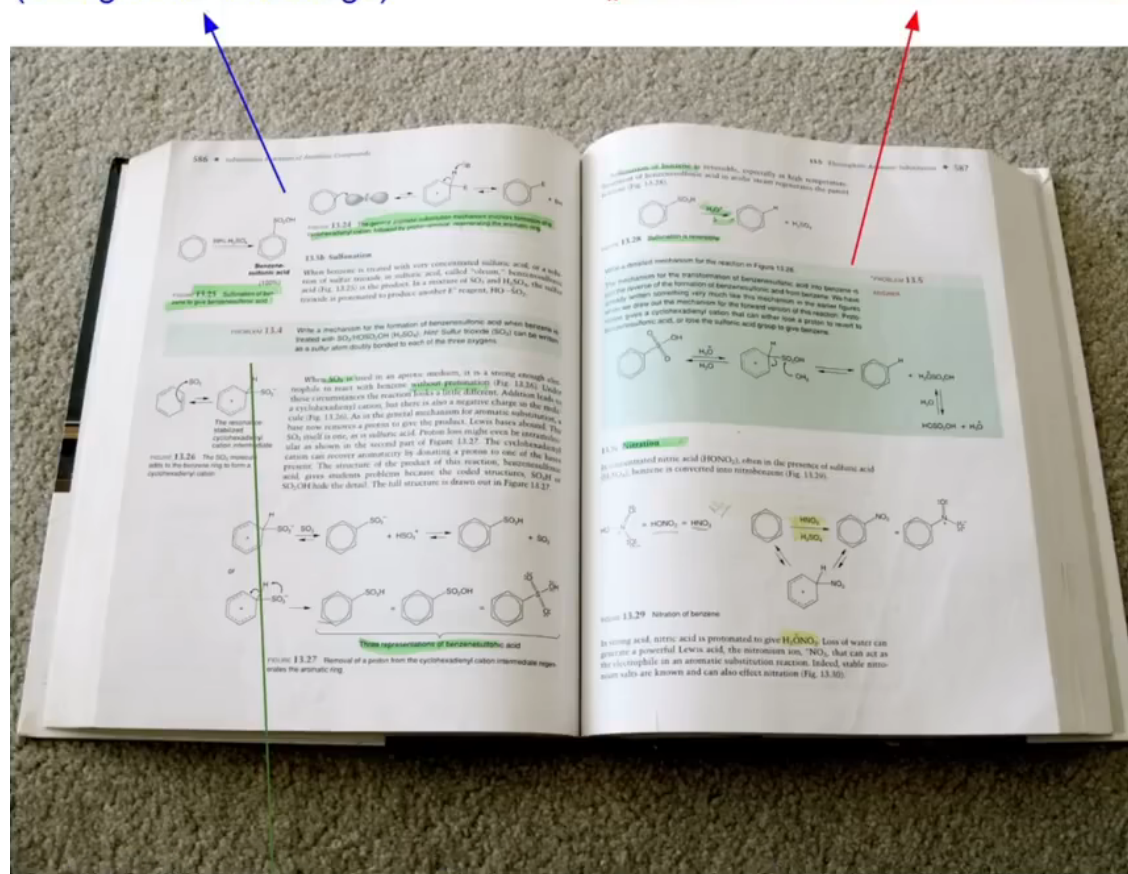
# 强化学习

在某些资料上 强化学习被归类为后训练的一部分 但是这步训练的方式是完全不同的方法

先粗略的介绍一下这种训练方法

exposition  $\Leftrightarrow$  pretraining  
(background knowledge)

worked problems  $\Leftrightarrow$  supervised finetuning  
(problem + demonstrated solution, for imitation)



practice problems  $\Leftrightarrow$  reinforcement learning  
(prompts to practice, trial & error until you reach the correct answer)

蓝色字是背景知识,大量的文本(预训练后的base model)

红色字是问题和回答(学会对话后的SFT model)

绿色字是课后习题

强化学习的意义就是通过记忆背景知识和模拟专家对话 对于大量问题尝试回答

对于一个问题 例如:

Emily buys 3 apples and 2 oranges. Each orange costs \$2. The total cost of all the fruit is \$13. What is the cost of each apple?

我们能够给出许多解决方案 我们并不知道那些方案对LLM来说 是可以接受的 所以

我们会让模型模拟解决方案 然后查看 有两个目的

1.答案对不对

2.回答是否可以让人类感到满意(解题过程)

模型是个随机系统 每一次的回答都会有所不同 例如:答案有可能错误 答案正确解题过程人类不满意...

示例只是一个简单的问题 所以基本上答案都正确

假设

现在提出一个困难的问题有15次回答 正确的回答有5次

其中有一次回答很好 也许是更快 或者其他什么标准

那么产生这次回答的词元就是我们需要的 我们希望在这种类型的提示词下 模型完成更多问题

我们鼓励模型的这种行为 模型就会自己思考 为什么这种回答更好 相关词元权重提高 下一次的回答路径 就会倾向选择这条路径 全过程没有人类参与

这就是强化学习大概的训练思路

这是deepseek-R1模型关于强化训练方向的文章

<https://arxiv.org/pdf/2501.12948>

里面主要讨论了强化训练在大语言模型中的重要性以及如何激发模型的推理能力

在经历强化训练之后 你会发现模型使用的词元更多了 响应的时间变得更长了

---

Question: If  $a > 1$ , then the sum of the real solutions of  $\sqrt{a - \sqrt{a + x}} = x$  is equal to

---

Response: <think>  
To solve the equation  $\sqrt{a - \sqrt{a + x}} = x$ , let's start by squaring both ...  
 $(\sqrt{a - \sqrt{a + x}})^2 = x^2 \implies a - \sqrt{a + x} = x^2$ .  
Rearrange to isolate the inner square root term:  
 $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$   
...  
**Wait, wait. Wait. That's an aha moment I can flag here.**  
Let's reevaluate this step-by-step to identify if the correct sum can be ...  
We started with the equation:  
 $\sqrt{a - \sqrt{a + x}} = x$   
First, let's square both sides:  
 $a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$   
Next, I could square both sides again, treating the equation: ...  
...

---

这是因为模型会重新评估步骤

我们根本不知道要给模型什么参数才能产生这种效果 这只能在强化学习中被发现

模型学会了思维链或者说 模型学会了思考 他可以对一个问题从不同角度解答, 这是一种优化的特性

我们将这个模型称为 推理模型(reasoning moble)

现在又产生了一个问题 我们刚才强化学习的目标产生的答案很简单 很轻易就能判断

那如果在不可验证领域 给不同的答案评分非常困难 这个策略又该如何实现

这个领域一般是创造性的 例如写诗歌 讲笑话

强化学习每次产生的内容有上千个结果 我们不可能使用人类来完成 我们需要一种自动化策略

<https://arxiv.org/pdf/1909.08593>

这篇论文提出了一种解决方法 **基于人类反馈的强化学习(RLHF)**

作者训练了一个模型 这个模型模仿人类的评分 这个模型叫奖励模型

他会对于模型强化学习后的作品进行评分和排序 例如创作的诗歌

当然 他也有缺点

1.我们的建立模型仅仅是模拟人类 他有自己的损失函数 这个有损的模拟会产生误导

2.强化学习非常擅长获得奖励 在前几百步诗歌的创造有改善 但突然急速下降 最后的输出会非常荒谬 但奖励模型的评分非常高 (有点像作文要写到得分点 结果最后全是得分点 无法构成完整的文章)

这种荒谬的输出被称为 对抗性例子 这里不详细介绍

## 链接

大模型介绍 3小时左右(建议完整看一遍)

<https://www.youtube.com/watch?v=7xTGNNLPyMI>

token数量查看器

<https://miniwebtool.com/zh-cn/ai-token-%E8%AE%A1%E6%95%B0%E5%99%A8/>

MCP介绍

<https://modelcontextprotocol.io/>

FineWeb数据集

<https://huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1>

词元表示 tiktokenizer

<https://tiktokenizer.vercel.app/>

生产环境神经网络结构 可视化模型

<https://bbycroft.net/llm>

gpt2论文 (pre trained transformer)

[https://cdn.openai.com/better-language-models/language\\_models\\_are\\_unsupervised\\_multitask\\_learners.pdf](https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf)

模型训练

[github.com/karpathy/llm.c/discussions/677](https://github.com/karpathy/llm.c/discussions/677)

大型服务器租用

<https://lambdalabs.com>

hyperbolic模拟器

<https://app.hyperbolic.xyz/models/llama31-405b-base-bf-16>

微调语言模型进行对话

<https://arxiv.org/pdf/2203.02155>

SFT数据集

[github.com/thunlp/UltraChat?tab=readme-ov-file](https://github.com/thunlp/UltraChat?tab=readme-ov-file)

Deepseek-R1 强化训练

<https://arxiv.org/abs/2501.12948>